Borrowing Strength Across Pairs: A Random-Effects Predictor on the Evaluator's Global Label List for the Predictive AI Evaluation Competition

Neel Sirivara
University of Washington
nsiriv@cs.washington.edu

Abstract

The Predictive AI Evaluation Competition (PAIEC) asks for the probability that an AI system (the *subject*) answers a benchmark *item* correctly, given visible attributes, the item text, and labels acquired at budgets of 0–31 per subject–benchmark pair; the score is a Brier area under the learning curve (ALC). The official streaming client passes *one global list* with the labels acquired for *all* sampled pairs to every prediction, whereas our earlier submission used only the target pair's labels. We now fit a random-effects logistic model to that list once per budget checkpoint: a hand-set attribute prior as offset, benchmark, subject and subject–benchmark effects, and an exact-item effect learned from other subjects' labels on the same item, with probit integration of the posterior uncertainty. Its prior standard deviations were chosen by nested, benchmark-held-out simulation of the official protocol on the official training release (16,718 pairs, 144 benchmarks), with design and ship rule pre-registered. In formative-sized test sets (at most 1,000 subject–item pairs) the ALC falls from 0.2095 (our earlier predictor, as the evaluator would run it) to 0.1966, a paired difference of -0.0129 [-0.0151, -0.0109] (cluster bootstrap over 135 benchmarks); it is -0.0125 when cross-pair evidence only comes from different models and -0.0289 when all pairs of a benchmark are scored together. Benchmark difficulty and exact-item difficulty contribute about equally. On a 17-pair testbed of four small open models the difference is -0.0074 [-0.0203, +0.0032] for the pre-registered acquisition trajectory (not significant) and significant in 4 of 5 trajectories. An acquisition rule that skips items already labelled for another subject changes the ALC by a further -0.0005 [-0.0009, -0.0002]. The predictor is standard-library Python; one fit of a 62,000-label list takes 1.68 s and a prediction 0.116 ms. Earlier negative results (a fitted attribute prior, a pre-registered item-text prior) are summarised. Code and text were produced with an LLM coding assistant (Claude).

1 Introduction

Evaluating every new AI system on every benchmark is expensive, and the number of benchmarks keeps growing [Liang et al., 2023]. Recent work therefore predicts performance instead of measuring it: from a few hundred informative items selected with item response theory (IRT) [Polo et al., 2024, Kipnis et al., 2025, Perlitz et al., 2024], adaptively per model [Hofmann et al., 2025, Zhuang et al., 2025], from other models' scores [Ruan et al., 2024, Ye et al., 2023], or from a few labelled instances of the new model [Pacchiardi et al., 2024]. PAIEC [Stanford AIMS Lab, 2026] turns this into a forecasting task with a precise interface: `predict([subject, item], labeled)` returns $\Pr(\text{correct})$, and `labeled` holds the labels acquired so far, at budgets of $n \in \{0, 1, 3, 7, 15, 31\}$ labels per subject–benchmark pair.

Technical report, Predictive AI Evaluation Competition (PAIEC), 40th Conference on Neural Information Processing Systems (NeurIPS 2026).

Our submission for rounds 1–3 read `labeled` as the target pair's own labels: a hand-set attribute prior, beta-binomial shrinkage of the pair rate [Efron and Morris, 1975], and item-level corrections gated by a leave-one-out test. It beat the organisers' empirical mean on two local testbeds and on the training release, while two attempts to improve the budget-0 prior with the release—a fitted attribute prior and a pre-registered item-text prior—failed under benchmark-held-out cross-validation. The diagnosis was that the residual error is dominated by *benchmark difficulty*, which nothing visible at budget 0 reveals. Reading the official streaming client closely (Section 2) shows that the information is there from budget 1 on: `labeled` is one *global* list with the labels of all sampled pairs. Other subjects' labels on the same benchmark reveal its difficulty, a subject's labels on other benchmarks its ability, and because acquisition and evaluation pools are split per pair, the very item being predicted is often labelled for another subject. The rules allow exactly this ("the submitted program may use the attributes and acquired labels supplied through the evaluation interface to adapt during evaluation").

Contributions. (1) A precise statement of the evaluator's global-evidence protocol and a faithful multi-pair simulator of it on the official training release, with benchmark-held-out folds and five test-set configurations. (2) A random-effects (Rasch-style) predictor fitted to the global list once per checkpoint [Lord, 1980, Breslow and Clayton, 1993], with an exact-item effect and probit integration [MacKay, 1992]; pre-registered selection and ship rule; paired, cluster-bootstrapped gains at every budget; component ablations; a check that the shipped files reproduce the simulator inside the organisers' own client. (3) A coverage-oriented acquisition rule. (4) The earlier per-pair predictor, kept as fallback and for budget 0, and its negative results (Appendices B–D).

Related work and design choices. Item response models—a subject ability and an item difficulty on a logit scale—are widely used for LLM benchmark data, to rank systems, find informative items and estimate full-benchmark scores from few items [Rodriguez et al., 2021, Polo et al., 2024, Kipnis et al., 2025]. These methods fit item parameters offline on hundreds of models; in PAIEC the test benchmarks are new, so difficulty must be learned *during* evaluation from a few labels of a few subjects. We therefore keep the Rasch structure but replace free parameters by random effects with Gaussian priors (a generalised linear mixed model [Breslow and Clayton, 1993]), which shrinks a benchmark's difficulty towards the prior when it has one or three labels and lets it speak when it has hundreds, and we use one difficulty per item only where the item itself was labelled. Adaptive testing [Hofmann et al., 2025, Zhuang et al., 2025] chooses the next item from the current ability estimate; the PAIEC stream only lets a policy accept or skip a random candidate, and uncertainty-based skipping did not help in round 1 (Appendix E), so our acquisition rule only maximises coverage of distinct items. Work that predicts a model's scores from other models' scores [Ruan et al., 2024, Ye et al., 2023] motivates pooling across subjects; our model does this at the item level and inside one evaluation run.

2 Task, interface and the global label list

A *subject* is a dictionary of strings (`normalized_name`, `provider`, `release_date`, `access_date`, `harness`, `harness_version`, `reasoning_effort`, `subject_features_extra`); an *item* has `item_content`, `item_features`, `interactors` and an anonymous `benchmark_id`. Each subject–benchmark pair with at least 80 distinct items is split 50/50 into an acquisition and an evaluation pool, fixed across submissions. Acquisition candidates of a pair arrive in a fixed random order; an optional `acquisition_function` returns `True` to reveal a label (31 per pair at most), otherwise labels are drawn uniformly. After 0, 1, 3, 7, 15 and 31 labels per pair every evaluation item is predicted, giving Brier scores [Brier, 1950] $B_0, \dots, B_{31}$ and

$$\mathrm{ALC} = 0.1\,B_0 + 0.2\,B_1 + 0.2\,B_3 + 0.2\,B_7 + 0.2\,B_{15} + 0.1\,B_{31}, \tag{1}$$

averaged over pairs (lower is better). A formative evaluation samples at most 1,000 subject–item pairs (acquisition and evaluation pools together) from benchmarks of the private test pool; runs have 8 hours.

The global label list. In the organisers' client (`tools/streaming_ingestion.py`, `run_streaming`) every coordinator event carries one list, `labeled = event["labeled"]`; at an `evaluate` event all distinct inputs of the checkpoint are predicted with `prediction_stream(model_path, unique, labeled, ...)`, i.e. with the *same* list, and the

competition page states that `labeled` "contains [[subject, item], label] entries across sampled subject–benchmark pairs". At budget n the list therefore holds up to n labels of *every* pair; at budget 0 it is empty. Our rounds 1–3 predictor, like the organisers' empirical-mean baseline, used only the target pair's labels; it selected them by benchmark and *model name*, so as deployed it also pooled the labels of other subjects with the same name (other reasoning efforts, harnesses or test conditions). Using the other entries is permitted; what is forbidden is extracting test data or trying to identify the anonymous subjects and benchmarks, and we do neither: the model only uses anonymous keys (the subject's full attribute dictionary, `benchmark_id`, item text) inside one evaluation run and keeps nothing across runs.

3 Method

Budget-0 prior (unchanged). $o(s,i) = \mathrm{clip}(z_{\mathrm{subj}}(s) + z_{\mathrm{item}}(i), \pm 1.5)$, a hand-set logistic form: release year, reasoning effort, parameter count or size words in the name, and harness for the subject; tier/level/points cues in `item_features` and item length for the item (Appendix B). At budget 0 the prediction is $\sigma(o)$.

Cross-pair random-effects model. For a label y_e of subject s on item i of benchmark b in the global list, stage 1 is the logistic random-effects model

$$\mathrm{logit}\ \Pr(y_e = 1) = o(s,i) + \mu + v_b + u_s + w_{sb}, \tag{2}$$

with independent priors $\mu \sim \mathcal{N}(0, \tau_\mu^2)$, $v_b \sim \mathcal{N}(0, \tau_b^2)$, $u_s \sim \mathcal{N}(0, \tau_s^2)$, $w_{sb} \sim \mathcal{N}(0, \tau_{sb}^2)$, subjects identified by their complete attribute dictionary (as the organisers' baseline does) and benchmarks by `benchmark_id`. We compute the MAP estimate by backfitting: one Newton step per effect type and sweep, $\theta_g \leftarrow \theta_g + \big(\sum_{e\in g}(y_e - p_e) - \theta_g/\tau^2\big)/\big(\sum_{e\in g} p_e(1-p_e) + 1/\tau^2\big)$, at most 25 sweeps (penalised quasi-likelihood for a GLMM [Breslow and Clayton, 1993]). The target's latent logit $m = o + \hat\mu + \hat v_b + \hat u_s + \hat w_{sb}$ (zero for groups without labels) has approximate posterior variance $V = \sum_g 1/(H_g + \tau_g^{-2})$, with τ_g^2 for an unseen group, and $p = \sigma\big(m/\sqrt{1 + \pi V/8}\big)$ [MacKay, 1992]. *Exact-item effect (stage 2).* If the target item (same `benchmark_id` and text) was labelled for other subjects, an item effect $d \sim \mathcal{N}(0, \tau_d^2)$ is estimated from those labels by 4 Newton steps on the conditional scale (their stage-1 logits multiplied by $c = \sqrt{1 + \pi\tau_d^2/8}$, since stage 1 averages over item difficulty), and $p = \sigma\big((c\,m' + \hat d)/\sqrt{1 + \pi\,\mathrm{Var}(d)/8}\big)$ with m' the probit-scaled stage-1 logit; without matches this reduces to the stage-1 prediction. The model also has slots for a model-name effect and a name–benchmark effect, which the selection switched off. Shipped prior standard deviations: $\tau_\mu = 0.5$, $\tau_b = 1.5$, $\tau_s = 0.5$, $\tau_{sb} = 0.6$, $\tau_d = 2.25$ (Section 4).

Engineering. `labeled` is identical for all calls of a checkpoint, so the parsed list and the fit are cached at module level under a fingerprint (length and a hash of about 70 evenly spaced entries) and replaced atomically under a lock; separate worker processes fit once each. A list of 62,000 labels is fitted in 1.68 s; a cache-hit call takes 0.116 ms (Table 4). The code is standard-library Python, deterministic, and falls back to the rounds 1–3 predictor on any failure.

Fallback: the per-pair predictor of rounds 1–3. Beta-binomial shrinkage of the pair rate towards the prior ($\alpha_0 = 4$ pseudo-counts) and, from 5 labels, a TF-IDF nearest-neighbour correction and (from 7 labels) a ridge adjustment on item features, each switched on only when a leave-one-out test on the pair's labels shows an improvement of 2 standard errors (Appendix B). It handles budget 0 and any failure of the cross-pair model.

Acquisition: fresh first. With cross-pair learning, a label on an item nobody has labelled can help every other subject whose evaluation pool contains that item, whereas a second label on an already labelled item adds less. `labeling.py` therefore accepts candidates in stream order (a uniformly random sample, as the default policy) except that it skips a candidate whose item is already in the global list for another subject while labels_remaining $<$ items_remaining, so the budget is always spent. The function has no `prediction` parameter, so the client does not call `predict` for every candidate.

Table 1: Brier ALC by test-set configuration (Section 4); units are (test set, pair), cluster-bootstrap 95% CIs over benchmarks. S0: our rounds 1–3 predictor as the evaluator runs it (global list, name-based pair filter); S1: the same with the target pair's labels only; XP: cross-pair model (fold-wise nested selection); oracle: each pair's evaluation success rate as a constant.

Configuration	units/bench.	S0	S1	XP	oracle	XP − S0 [95% CI]	XP − S1 [95% CI]
A: formative-like (primary)	13,460/135	0.2095	0.2074	0.1966	0.1731	-0.0129 [-0.0151, -0.0109]	-0.0108 [-0.0134, -0.0084]
E: distinct models	8,673/135	0.2078	0.2074	0.1953	0.1730	-0.0125 [-0.0148, -0.0100]	-0.0121 [-0.0146, -0.0095]
B: one benchmark, all pairs	6,772/135	0.2131	0.2087	0.1842	0.1793	-0.0289 [-0.0342, -0.0239]	-0.0245 [-0.0312, -0.0181]
C: one fold, many benchmarks	9,577/125	0.2190	0.2084	0.1797	0.1780	-0.0393 [-0.0521, -0.0280]	-0.0287 [-0.0384, -0.0191]
D: A + eval-only small pairs	14,293/135	0.2097	0.2096	0.1969	0.1706	-0.0128 [-0.0150, -0.0108]	-0.0127 [-0.0160, -0.0094]

4 Evaluation protocol

Frame. The official training release (`aims-foundations/measurement-db`, revision `54074453da`) gives 16,718 pairs of (subject, test condition) on 144 benchmarks with at least 20 items (16,303 with 80), 3,118,742 item-level labels (mean over trials) and 165,764 distinct items. Each pair keeps its 200 answered items with the lowest benchmark-level hash rank, so subjects of a benchmark share items as far as their answered sets allow. The release is dominated by test-condition variants of one model: 92.7% of pairs share (model name, benchmark) with another pair.

Simulator. The five benchmark folds of round 2 hold out whole benchmarks. A *test set* is a set of pairs of one fold that shares one global list. Per pair (fixed across test sets) items are split 50/50 and the acquisition pool is permuted at random; budget n is its first n labels (the official random policy yields a uniformly random subset in stream order); at budget n every prediction of the test set receives the union of all its pairs' first n labels. Configurations: **A** (primary, formative-like): 1–3 random benchmarks, 10 random pairs with at least 80 items split over them, 100 items per pair ($\leq$1,000 subject–item pairs), 300 test sets per fold; **E**: as A but no two pairs share a model name, so cross-pair evidence comes only from different models; **B**: all pairs of one benchmark (up to 300, 200 items each); **C**: up to 2,000 pairs of a whole fold in one list; **D**: A plus up to two pairs with 20–79 items that are evaluated on all items and never acquired (the page does not say how such pairs are handled). Per-pair ALC; means over (test set, pair) units; 95% CIs by a cluster bootstrap over benchmarks (2,000 resamples).

Pre-registration and selection. `experiments/results/round4/PREREG.md` (SHA-256 prefix `b9e95a5b47e9`, consistent with every result file) fixed configurations A–D, predictors, the grid of 864 settings (τ_b, τ_{name}, τ_s, $\tau_{\text{name},b}$, τ_{sb}, τ_d, probit on/off; $\tau_\mu = 0.5$), nested selection (for each outer fold, the best setting on the other four folds' config-A test sets), the primary outcome and the ship rule before any outcome was computed: ship iff (a) the config-A ALC difference to our earlier predictor as deployed (S0) has a CI below zero, (b) the real testbed (Section 5) is not significantly worse, (c) no other configuration is significantly worse, and (d) engineering gates (never raises, smoke test and validator pass, a 62,000-label checkpoint in $\leq$60 s). The selected setting lay on the grid boundary in τ_b and τ_d; an addendum written after the primary result (consistent) added configuration E and an extended grid of 2,160 settings, selected on A and E jointly; it returned the same setting (Appendix A). The stdlib `model.py` and the vectorised simulator agree to 1.4×10^{-5} on 56,750 predictions.

Local testbeds. The real testbed has 17 pairs: four small open models (Qwen2.5-Instruct 1.5B/3B/7B, Llama-3.2-3B) run locally with Ollama on GSM8K, SVAMP and a BIG-Bench-Hard mix under two harnesses (Appendix C); for round 4 all pairs share one global list. A synthetic IRT testbed was used in rounds 1–3 only.

5 Results

Main result. Table 1. In the primary configuration A the cross-pair model reaches 0.1966 against 0.2095 for our earlier predictor as deployed: -0.0129 [-0.0151, -0.0109], significant, lower ALC on 73% of units, and 35% of the gap between S0 and the pair-rate oracle closed; per outer fold it ranges from -0.0167 to -0.0074 (Appendix A). Rule (a) holds. The gain holds where cross-pair evidence only comes from different models (E: -0.0125 [-0.0148, -0.0100]), grows when more pairs share the list

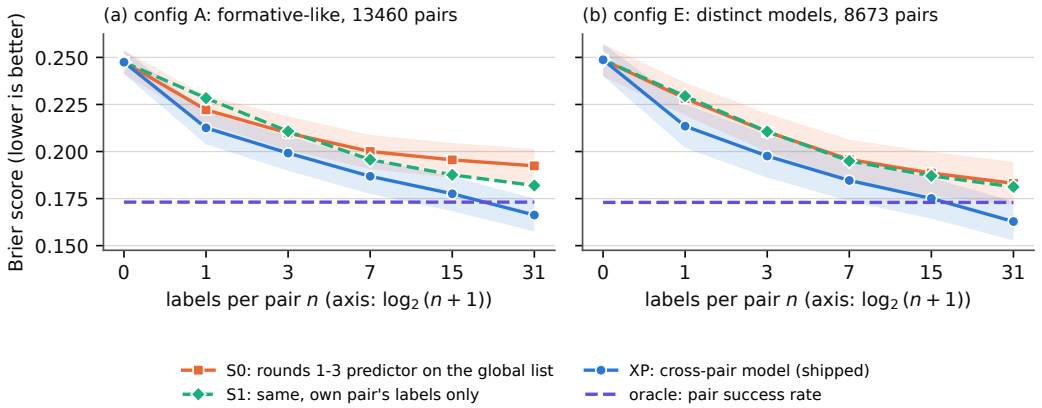


Figure 1: Brier score per budget on the training release (benchmark-held-out). Bands: marginal 95% cluster-bootstrap CIs over benchmarks; the paired differences, which are much tighter, are in Table 2. XP falls below the pair-rate oracle at $n = 31$ because it learns item difficulty from other subjects.

Table 2: Brier score per budget and ALC (configurations A and E) and the paired difference XP − S0 with its 95% cluster-bootstrap CI.

	Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
A	S0 (rounds 1–3, as deployed)	0.2475	0.2221	0.2098	0.2000	0.1956	0.1924	0.2095
A	S1 (own pair's labels only)	0.2475	0.2284	0.2107	0.1957	0.1876	0.1819	0.2074
A	XP (cross-pair, shipped)	0.2475	0.2126	0.1992	0.1868	0.1776	0.1663	0.1966
A	oracle: pair rate	0.1731	0.1731	0.1731	0.1731	0.1731	0.1731	0.1731
A	XP − S0	–	-0.0096	-0.0106	-0.0132	-0.0180	-0.0261	-0.0129
	95% CI	–	[-0.0131, -0.0063]	[-0.0133, -0.0082]	[-0.0156, -0.0109]	[-0.0211, -0.0151]	[-0.0301, -0.0224]	[-0.0151, -0.0109]
E	S0 (rounds 1–3, as deployed)	0.2487	0.2282	0.2104	0.1958	0.1886	0.1831	0.2078
E	S1 (own pair's labels only)	0.2487	0.2294	0.2106	0.1950	0.1871	0.1812	0.2074
E	XP (cross-pair, shipped)	0.2487	0.2135	0.1976	0.1846	0.1751	0.1628	0.1953
E	oracle: pair rate	0.1730	0.1730	0.1730	0.1730	0.1730	0.1730	0.1730
E	XP − S0	–	-0.0147	-0.0128	-0.0111	-0.0135	-0.0203	-0.0125
	95% CI	–	[-0.0207, -0.0087]	[-0.0161, -0.0094]	[-0.0135, -0.0085]	[-0.0167, -0.0102]	[-0.0250, -0.0154]	[-0.0148, -0.0100]

(B: -0.0289; C: -0.0393), and covers pairs that were never acquired (D, evaluation-only pairs: -0.0120 [-0.0217, -0.0036]); rule (c) holds. Against S1, i.e. against a correct per-pair filter, the gains are -0.0108 (A) to -0.0287 (C), so they are not an artefact of S0's name pooling; S0 and S1 themselves do not differ significantly in A (S1 − S0: -0.0021 [-0.0050, +0.0009]), but name pooling hurts when a whole fold shares the list (C: -0.0106 [-0.0168, -0.0025]).

Per budget. Table 2 and Figure 1. Budget 0 is unchanged (the list is empty). The difference is significant at every budget from $n = 1$ (-0.0096 [-0.0131, -0.0063] in A), where the benchmark effect is learned from the other pairs' first labels, and largest at $n = 31$ (-0.0261 [-0.0301, -0.0224]), where many evaluation items have been labelled for another subject; at $n = 31$ XP (0.1663) is below the pair-rate oracle (0.1731), the floor of any predictor without item-level information.

Where the gain comes from. Table 3. Removing the benchmark effect costs +0.0048 [+0.0032, +0.0064] (A) and removing the exact-item effect +0.0048 [+0.0038, +0.0059]: benchmark difficulty, invisible at budget 0, and item difficulty, invisible from 31 own labels, contribute about equally. The subject–benchmark effect (+0.0010), the subject effect (+0.0004) and probit integration (+0.0005) are small; the subject effect rarely transfers because full attribute dictionaries rarely repeat across benchmarks (211 of 16,378 do). Benchmark and pair effects alone recover more than half of the gain (their ALC is 0.2018 against 0.2095 for S0), whereas name/subject and pair effects (0.2095) or item and pair effects (0.2101) alone recover none of it: in this model the item effect becomes useful only together with the benchmark effect, which puts the other subjects' labels on a common scale. The extended grid places the optimum inside: the best ALC per τ_d is 0.2013, 0.1969, 0.1965, 0.1968, 0.1980 for $\tau_d = 0, 1.5, 2.25, 3, 4$ (A), and per τ_b 0.1966, 0.1965, 0.1966 for $\tau_b = 1, 1.5, 2$.

Table 3: Components (all-fold setting; configurations A and E): ALC and paired difference to the shipped setting (positive: removing the component hurts).

Variant	A: formative-like ALC	A: formative-like Δ [95% CI]	E: distinct models ALC	E: distinct models Δ [95% CI]
shipped setting (all effects)	0.1965	–	0.1952	–
− benchmark effect	0.2013	+0.0048 [+0.0032, +0.0064]	0.2003	+0.0050 [+0.0032, +0.0070]
− subject effect	0.1969	+0.0004 [+0.0001, +0.0008]	0.1954	+0.0002 [-0.0002, +0.0006]
− subject–benchmark effect	0.1975	+0.0010 [+0.0004, +0.0017]	0.1959	+0.0006 [-0.0000, +0.0014]
− exact-item effect	0.2013	+0.0048 [+0.0038, +0.0059]	0.2005	+0.0053 [+0.0038, +0.0068]
− probit integration	0.1970	+0.0005 [+0.0000, +0.0009]	0.1958	+0.0006 [-0.0000, +0.0011]
benchmark + pair effects only	0.2018	+0.0053 [+0.0043, +0.0063]	0.2008	+0.0056 [+0.0041, +0.0071]
name/subject + pair effects only	0.2095	+0.0130 [+0.0105, +0.0157]	0.2095	+0.0143 [+0.0114, +0.0173]
pair + item effects only	0.2101	+0.0135 [+0.0099, +0.0174]	0.2096	+0.0144 [+0.0100, +0.0189]
pair effect only	0.2167	+0.0202 [+0.0163, +0.0243]	0.2168	+0.0216 [+0.0171, +0.0264]

Table 4: Runtime of the shipped `model.py` on one laptop core (synthetic global lists of 250-word items, 31 labels and 50 evaluation items per pair): first call of a checkpoint (parse and fit), later calls, and the whole checkpoint.

Pairs	Labels	Predictions	Fit (s)	Per call (ms)	Checkpoint (s)
10	310	500	0.01	0.102	0.1
100	3,100	5,000	0.07	0.094	0.5
1,000	31,000	50,000	0.56	0.098	5.5
2,000	62,000	100,000	1.68	0.116	13.3
4,000	124,000	200,000	3.68	0.128	29.3

Real testbed. With one global list over the 17 real pairs (seed 0, as pre-registered), XP reaches 0.2222 against 0.2295 for our earlier predictor as deployed: -0.0074 [-0.0203, +0.0032] (not significant; rule (b) holds); over five acquisition trajectories the difference ranges from -0.0138 to -0.0074 and is significant in 4. The deployed name filter pools the two harnesses of the same model; a per-pair filter would change its ALC by -0.0071 [-0.0164, +0.0009] (not significant); against that per-pair predictor XP changes the ALC by -0.0002 [-0.0072, +0.0074] (seeds: -0.0047 to -0.0002). Three benchmarks with four to seven pairs each give little cross-pair evidence, and at $n = 1$ XP is worse here (0.2549 vs. 0.2468).

Acquisition. In configuration A the fresh-first rule changes the XP ALC by -0.0005 [-0.0009, -0.0002] (significant), essentially all of it at $n = 15$ (-0.0017) and $n = 31$ (-0.0017), where item matches matter; it does not affect S0 (+0.0001 [-0.0001, +0.0003]). On the real testbed (seed 0) it changes the ALC by +0.0001 [-0.0010, +0.0011] (not significant; over five seeds: mean -0.0019, significantly better in 2, significantly worse in 1). Under the pre-registered rule it is shipped.

Engineering. Table 4: even 4,000 pairs per checkpoint take well under a minute on one core (rule (d)). Inside the organisers' own streaming client, driven by a local mock coordinator on the real testbed, the shipped files run in `stream` acquisition mode with 0 `predict` calls during 1,135 acquisition decisions, and the resulting ALC equals the simulator's to machine precision. The smoke test adds 220 cross-pair cases (unseen subjects and benchmarks, malformed entries, determinism, four threads, a forced fit failure, a 62,000-label list). The ZIP (19.8 kB, SHA-256 prefix `c76331d084f1`) passes the official validator. *Decision: the cross-pair model and the fresh-first rule are shipped.*

Earlier rounds in brief. On the local testbeds the per-pair predictor beat the organisers' empirical mean (real: 0.2224 vs. 0.2871; synthetic: 0.1797 vs. 0.2208), mostly through shrinkage; its item-level components and the acquisition policies changed the ALC by less than 0.005; on the release a fitted attribute prior and a pre-registered item-text prior (-0.0000 [-0.0013, +0.0014]) did not beat the hand-set prior (Appendices C–H). If the evaluator passed only the target pair's labels, XP would still be slightly better on the release (-0.0016 [-0.0028, -0.0005] against S1) but worse on the real testbed (+0.0060 [+0.0022, +0.0099]).

6 Limitations

Test-set composition is unknown. The gain depends on how many pairs share a list and how they overlap; configurations A–E bracket plausible cases, and the gain is positive in all of them, but formative sets may be smaller or contain one pair per benchmark, where only the subject effect and the fallback-like pair shrinkage remain. *Release structure.* Most release pairs are test-condition variants of one model; configuration E removes that structure and the gain persists, but the selected τ_d was tuned on the mixture. Item identity is exact text match within a benchmark; paraphrased or condition-specific item texts give no match. *Pairs with fewer than 80 items* are handled under one assumed rule (D). *Real testbed.* 17 pairs of small models on three benchmarks give wide CIs; there the gain over a correct per-pair filter is not significant, and XP is worse at $n = 1$. *Budget 0* still rests on the hand-set prior, whose weaknesses for frontier subjects are analysed in Appendix I. *Acquisition* gains are small and mixed on the real testbed.

7 Reproducibility

Code, results and report: `https://github.com/neelsirivara/neurips2026-paiec`. `experiments/src/model.py` and `labeling.py` are the shipped files; `xpair_sim.py` builds the round-4 frame and runs the simulator, selection, real-testbed, acquisition, runtime and equivalence checks (`build, grid, cv, addendum, acq, realtest, realacq, perpair, runtime, equivalence`); `official_loop_check.py` runs the shipped files inside the organisers' client; `smoke_test.py` and `build_submission.py` check and package; earlier rounds' scripts are listed in Appendix C. `build_paper.sh` regenerates every number (`make_macros.py`), figure and the claim audit (`make_audit.py`) from the result files. Seeds are fixed. Hardware: one laptop CPU (8 cores, 34 GB RAM, no GPU); the round-4 simulations take about 20 minutes on four cores.

8 Disclosure

Training data: `aims-foundations/measurement-db` (revision `54074453da`; `subjects`, `items` and `responses` of the 163 competition-eligible directories, 144 with usable pairs; downloaded 2026-10-22 with the author's account). Round 4 used it to simulate the protocol and to select the prior standard deviations of the cross-pair model and the acquisition rule; earlier rounds used it to validate the per-pair rule and to fit two candidate priors that were not shipped. No other inventory source. The shipped files contain no data. *Evaluation data:* GSM8K, SVAMP and ten BIG-Bench-Hard subtasks, only to label local models. *Pretrained models:* Qwen2.5-Instruct 1.5B/3B/7B and Llama-3.2-3B-Instruct via Ollama as evaluation subjects; `all-MiniLM-L6-v2` in one unshipped ablation. No commercial API. *Compute:* a CPU-only laptop. *LLM coding assistant:* code, experiments and text were drafted by Anthropic's Claude (Claude Code) running autonomously on the author's machine; the author reviewed the pipeline and is responsible for the content.

References

N. E. Breslow and D. G. Clayton. Approximate inference in generalized linear mixed models. *Journal of the American Statistical Association*, 88(421):9–25, 1993. doi: 10.1080/01621459.1993.10594284.

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975. doi: 10.1080/01621459.1975.10479864.

Susan E. Embretson and Steven P. Reise. *Item Response Theory for Psychologists*. Psychology Press, 2000. doi: 10.4324/9781410605269.

Valentin Hofmann, David Heineman, Ian Magnusson, Kyle Lo, Jesse Dodge, Maarten Sap, Pang Wei Koh, Chun Wang, Hannaneh Hajishirzi, and Noah A. Smith. Fluid language model benchmarking. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2509.11106.

Alex Kipnis, Konstantinos Voudouris, Luca M. Schulze Buschoff, and Eric Schulz. metabench – a sparse benchmark of reasoning and knowledge in large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.12844.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. arXiv:2211.09110.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Routledge, 1980. doi: 10.4324/9780203056615.

David J. C. MacKay. The evidence framework applied to classification networks. *Neural Computation*, 4(5):720–736, 1992. doi: 10.1162/neco.1992.4.5.720.

Lorenzo Pacchiardi, Lucy G. Cheke, and José Hernández-Orallo. 100 instances is all you need: Predicting the success of a new LLM on unseen data by testing on a few instances, 2024. arXiv:2409.03563; presented at the KDD 2024 Workshop on Evaluation and Trustworthiness of Generative AI Models.

Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking of language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2308.11696.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.

Pedro Rodriguez, Joe Barrow, Alexander Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4486–4503, 2021. doi: 10.18653/v1/2021.acl-long.346.

Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.10938.

Stanford AIMS Lab. Predictive AI evaluation competition (NeurIPS 2026). Competition web page, 2026. URL `https://aims.stanford.edu/competition`. Accessed 2026-10-21.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL `https://github.com/aims-foundations/paiec_baseline`.

Qinyuan Ye, Harvey Yiyun Fu, Xiang Ren, and Robin Jia. How predictable are large language model capabilities? A case study on BIG-bench. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. arXiv:2305.14947.

Yan Zhuang, Qi Liu, Zachary A. Pardos, Patrick C. Kyllonen, Jiyun Zu, Zhenya Huang, Shijin Wang, and Enhong Chen. Position: AI evaluation should learn from how we test humans. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2306.10512.

A Round 4: details

Selection. The nested selection picked the all-fold setting in 3 of 5 outer folds; the per-fold config-A differences XP − S0 range from -0.0167 to -0.0074 (negative in 5 of 5). All-fold setting (shipped): $\tau_\mu = 0.5$, $\tau_b = 1.5$, $\tau_{\text{name}} = 0$, $\tau_s = 0.5$, $\tau_{\text{name},b} = 0$, $\tau_{sb} = 0.6$, $\tau_d = 2.25$, probit on; with it the config-A difference to S0 is -0.0130 [-0.0151, -0.0110] (nested estimate: -0.0129).

Amendment and addendum. One amendment was made to `PREREG.md` before any outcome was computed: the acquisition rule's skip threshold (registered as labels_remaining/items_remaining $<$ 0.5, which with 50-item pools would never skip) was replaced by the minimal one that still spends the budget. Addendum 1 was written at 20:58 PDT after the primary result and before any further outcome: configuration E, the extended grid ($\tau_b \in \{0.5, 1, 1.5, 2, 3\}$, $\tau_d \in \{0, 0.75, 1.5, 2.25, 3, 4\}$), and a "robust pick" minimising the average of the A and E ALCs, to be shipped only if not significantly worse than the pre-registered pick on A and better than S0 on E. It selected the same setting (5 of 5 folds; nested difference to the pre-registered nested XP on A -0.0001 [-0.0002, +0.0000]); shipped: robust pick (addendum 1). Best ALC per τ_d on E: 0.2005, 0.1979, 0.1957, 0.1952, 0.1955, 0.1964 for $\tau_d = 0, 0.75, 1.5, 2.25, 3, 4$.

Configurations. Units, benchmarks, test sets and labels at budget 31: A 13,460/135/1,500/417,260; E 8,673/135/1,500/268,863; B 6,772/135/135/209,932; C 9,577/125/5/296,887; D 14,293/135/1,500/417,260. In D the acquired pairs gain -0.0129 [-0.0151, -0.0109]; S0 gives evaluation-only pairs the mean of the whole list or of same-name pairs, XP their benchmark and subject effects. Frame structure: median 17.5 pairs and 3 model names per benchmark (maximum 4,278 pairs), 342 names on two or more benchmarks; 98.5% of labels are on an item that at least two pairs answered.

Checks. Pre-registered equivalence: 24 config-A test sets re-predicted by the stdlib `model.py` from dictionary inputs with binary labels: maximum absolute difference 1.4×10^{-5}. Real-testbed per-budget Brier (seed 0) for XP / S0-deployed / per-pair: B_1 0.2549/0.2468/0.2472, B_7 0.2100/0.2219/0.2118, B_{31} 0.1916/0.2130/0.1991. Fresh-first skipped 154,939 candidates in configuration A (ALC 0.1960 vs. 0.1965).

B Per-pair predictor (rounds 1–3; now the fallback and the budget-0 prior)

All constants below are hand-set and are still shipped in `model.py`: the budget-0 prior is used by the cross-pair model, the rest is its fallback. The few whose values were adjusted after looking at the local testbeds are marked † and have sensitivity analyses in Appendix D.

Budget-0 prior. A logistic prior $p_0 = \sigma(z)$ with $z = \mathrm{clip}(z_{\mathrm{subj}} + z_{\mathrm{item}}, \pm 1.5)$, so $p_0 \in [0.18, 0.82]$. The subject part is $z_{\mathrm{subj}} = 0.7 \cdot (0.35\,\mathrm{clip}(t - 2024.5, \pm 2) + e + 0.25\,\mathrm{clip}(\log_2(N/30\mathrm{B}), \pm 4) + 0.1\,\mathbb{1}[\text{non-default harness}])$, where t is the release year (fractional), $e \in \{0, 0.15, 0.3, 0.45\}$ encodes an empty, low, other or high `reasoning_effort`, and N is a parameter count parsed from the name ("7b", "70B"); when no count is present, name tokens such as *mini/nano/flash* contribute -0.3 and *pro/opus/ultra* contribute 0.2. An intercept (0) and a per-provider table (0 entries) complete the subject part; both are empty in the shipped prior and are the slots a fitted coefficient table would fill. The item part parses `key=value` pairs in `item_features`: a tier/difficulty/level value matching *hard/expert/...* adds -0.6, *easy/basic/...* adds 0.5, a numeric level $\ell \in [1, 5]$ adds $-0.25(\ell - 3)$, and points above one subtract 0.1 each (capped at five); finally the item length adds $-0.12\,\mathrm{clip}(\log_2(L/600), \pm 3)$ for L characters. The directions encode common knowledge (newer, larger, reasoning-enabled subjects do better; harder, longer, higher-value items are failed more often); the magnitudes were chosen to move the prior by at most a few tenths of a logit per attribute and were not tuned on either testbed.

Pair-level shrinkage. Given n acquired labels $y_1, \ldots, y_n$ of the same pair with item logits $z_{\mathrm{item}}^{(j)}$ and mean $\bar{z}$, the prior rate is $r = \sigma(\mathrm{clip}(z_{\mathrm{subj}} + \bar{z}))$ and the posterior mean of a Beta–binomial model with prior strength $\alpha_0 = 4^\dagger$ pseudo-counts is $\rho = (\alpha_0 r + \sum_j y_j)/(\alpha_0 + n)$ [Efron and Morris, 1975]. The pair estimate for a target item is $p_{\mathrm{pair}} = \sigma(\mathrm{logit}\,\rho + z_{\mathrm{item}} - \bar{z})$, i.e. the item cues shift the pair rate on the logit scale.

Item-level component 1: ridge features. From $n \geq 7$ labels, three cheap features of the item text—log length, digit density and log(1+number of sentences)—are standardised over the acquired items and a logistic regression with offset $\mathrm{logit}\,p_{\mathrm{pair}}$ and ridge penalty $\lambda = 4$ is fitted by six Newton steps; its adjustment $\mathrm{clip}(\beta^\top \tilde{f}, \pm 1)$ is added to the target's logit.

Item-level component 2: TF-IDF nearest neighbours. Item texts (content, features and interactors, truncated to 4000 characters) are tokenised into lower-case words and numbers; the acquired set defines the vocabulary and document frequencies, with sublinear term frequency, smoothed idf and ℓ_2 normalisation (bigram weight 1; target terms absent from the acquired vocabulary are dropped: no). With cosine similarities s_j between the target and acquired item j and weights $w_j = (\kappa s_j)^2$, the correction is a shrunken kernel-weighted residual,

$$p = \mathrm{clip}\Big(a + \frac{n}{n + N_0} \cdot \frac{\sum_j w_j\,(y_j - a)}{\tau + \sum_j w_j}\Big), \qquad N_0 = 4,\ \tau = 2, \tag{3}$$

where the anchor a is the pair estimate (optionally including the ridge adjustment) and $\kappa = 2$ by default. With at most 31 short documents most bigrams are unique, so cosine similarities are small (typically below 0.15 even for related items), which deliberately keeps the correction small unless items share many terms; Appendix E compares a sharper unigram-only representation.

Leave-one-out gating. From $n \geq 5$ labels, every configuration in $\{\text{ridge off, on}\} \times \{\kappa \in \{0, 1, 2, 4\}\}$ ($\kappa = 0$ means no kNN) is scored by its leave-one-out Brier score on the acquired labels themselves: label j is predicted from the other $n - 1$ labels, including a leave-one-out shrinkage estimate and a leave-one-out ridge fit. The best configuration replaces plain shrinkage only if its mean per-item improvement exceeds 2 standard error(s) of that mean[†]; otherwise the predictor returns p_{pair}. This makes adaptation *opt-in per pair and budget*: a pair whose labels show no exploitable structure is predicted by shrinkage alone. Below 5 labels no item-level component is used at all (in preliminary runs an always-on kNN from one or three labels only added noise at B_1 and B_3).

Acquisition policy. `labeling.py` implements the streaming interface and never raises. Candidates are accepted in stream order (uniformly random, hence the same label distribution as the evaluator's default) except that a candidate whose maximum TF-IDF cosine similarity to an already acquired item exceeds 0.6 is skipped while fewer than 90% of the remaining candidates are needed; it always accepts when the remaining candidates are needed to spend the budget. We compare this *novelty* filter with the evaluator's random policy, plain stream order, the organisers' uncertainty band $|p - 0.5| \leq 0.15$ [Truong et al., 2026], and a novelty-plus-uncertainty variant (skip $|p - 0.5| > 0.45$ once 7 labels are known).

Implementation (shipped files). Predictions are clipped to $[0.01, 0.99]$; `predict` is wrapped in a fallback chain (cross-pair model → per-pair full model → shrinkage only → smoothed label mean → 0.5), accepts labels given as booleans, numbers or strings, tolerates missing fields, malformed entries and non-dictionary inputs, and is deterministic. The submission ZIP (19.8 kB; members `README.md`, `labeling.py`, `model.py`) contains no `models.txt` and no `requirements.txt`: both files import only standard-library modules (`model.py`: `array`, `base64`, `hashlib`, `json`, `math`, `os`, `re`, `sys`, `threading`, `zlib`; the round-3 item-text-prior hooks stay inactive because no weight file is shipped; `labeling.py` loads its sibling `model.py` by path) and make no network calls. The organisers' `check_submission_zip.py` (commit `cc88a959`) passes on the ZIP. Our smoke test calls `predict` with the organisers' sample inputs at every budget and with malformed labels and inputs (44 cases), plus 220 cross-pair cases, and checks determinism; with the cross-pair model enabled the mean latency is 0.32 ms per call on short items and 0.06 ms at budget 31 with 300-word items (maximum 0.7 ms), about 16,449 calls per second on one laptop core.

C Rounds 1–3: data, testbeds and the training release

What we could and could not use. The official training data (`measurement-db`: item-level responses of many subjects on curated benchmarks) require accepting terms with a Hugging Face account. For most of the project we had no credentials and did not try to obtain the data in any other way; the per-pair predictor and all its constants were fixed before any of it was seen. Its public file listing (2026-10-22, revision `54074453da`) held 281 benchmark directories: 14.8 GB of `benchmarks`, `subjects`, `items` and `responses` tables plus 107.8 GB of traces and 154.7 GB of assets. On that day the author's login became available and we downloaded the 163 competition-eligible directories (binary, item-level responses; the eligibility flags live inside the `benchmarks` tables) without traces or assets, and used them as described next. We also inspected the organisers' inventory of eligible benchmarks (1,261 entries) but fitted nothing on any of them. The two local testbeds below only

evaluate design choices; the few constants they influenced are marked in Appendix B and probed in Appendix D.

The training release: validation and a fit that was not shipped. `fit_prior_from_measurement_db.py` builds one row per (subject, test condition, item) with the mean response over trials as label—16,718 pairs with at least 20 distinct items from 144 benchmarks, 3,118,742 rows (pairs capped at 200 items; 16,303 pairs have the 80 items the protocol requires)—and does two things (Appendix G). First, it evaluates the rounds 1–3 prior-plus-shrinkage rule with the protocol in closed form (50/50 split, 31 random acquisitions, Eq. 1; without the item-level components, which change the ALC by < 0.001 locally): ALC 0.2051 [0.2042, 0.2060] against 0.2396 [0.2380, 0.2411] for the organisers' empirical mean and 0.2470 for the constant, with $\alpha_0 = 4$ the best of $\{2, 4, 8\}$. The ordering and the size of the gain measured locally thus transfer to real systems on real benchmarks. Second, it fits an L_2-penalised logistic regression on exactly the feature terms of the hand-set prior plus a provider table and benchmark fixed effects (nuisance parameters; the shipped intercept is their mean), pairs weighted equally, with five-fold cross-validation over whole benchmarks. The fitted prior *loses* to the hand-set one on held-out benchmarks at every penalty (budget-0 Brier 0.2421–0.2455 against 0.2388; paired differences at the selected setting +0.0033 [+0.0030, +0.0036] at budget 0 and +0.0009 [+0.0008, +0.0009] in ALC), and an intercept-only prior is worse still (0.2519). The ship-only-if-better rule therefore kept the hand-set constants; in rounds 1–3 no coefficient of the submission derived from the release (round 4 selects the cross-pair model's prior standard deviations on it, Section 4). Where the data speak, the fitted signs agree with the hand-set values; what held-out benchmarks punish is the provider table (Appendix G).

An item-text difficulty prior (pre-registered; not shipped). Benchmark difficulty dominates the pair rate (benchmark means have standard deviation 0.223) and the attribute prior cannot see it. We therefore tested, with design and ship rule fixed in `PREREG.md` (SHA-256 `bb54af425d...`) before any fit, whether the item text predicts difficulty on *unseen* benchmarks: a ridge regression on 262,144 signed-hashed buckets (words, bigrams, character trigrams, `item_features`) and 16 text statistics (length, LaTeX, code, answer choices, ...), trained on the item logit the hand-set prior leaves unexplained (109,233 items, benchmarks weighted equally) and stacked onto the prior as $T = Ws + B$ with W, B fitted on out-of-fold scores; shrinkage is unchanged. On the same benchmark-held-out folds, every choice nested inside the training benchmarks, the ALC changes by -0.0000 [-0.0013, +0.0014] (cluster bootstrap over 135 benchmarks), closing -0.1%, 0.1% and 0.1% of the gap to the pair-rate oracle at $n = 0, 1, 3$. The inner CV chose the largest penalty of the grid in 5 of 5 folds and the out-of-fold slope W was *negative* in all 5 (-0.32 to -0.03): text patterns learned on some benchmarks point the wrong way on others. Exploratory variants registered afterwards (penalties up to 10^4; statistics alone) give +0.0002 and +0.0001, the local testbeds +0.0019 (real, not significant) and +0.0003 (synthetic, not significant). Headroom exists—knowing each held-out benchmark's mean rate would gain -0.0061 ALC—but item text in this form does not reach it; the rule fails and the submission is unchanged (Appendix H).

Real Ollama testbed. To obtain item-level correctness labels of real language models we ran four small open-weight models locally with Ollama (Q4_K_M quantisation, greedy decoding, CPU only): Qwen2.5-Instruct 1.5B, 3B and 7B and Llama-3.2-3B-Instruct. Three public benchmarks were used, none of which appears in the PAIEC inventory (only a derived variant, `mr_gsm8k`, does): the first 200 GSM8K test items, the first 200 SVAMP test items (with the SVAMP problem type as `item_features`), and a 300-item mix of ten BIG-Bench-Hard subtasks (30 items each, subtask name as `item_features`). Two harnesses are represented: a direct-answer prompt ("reply with only the final answer"; exact match after numeric or option normalisation) and, for GSM8K/SVAMP, an earlier 8-shot chain-of-thought run of the Qwen models whose greedy answers we re-used. Each (model, harness, benchmark) triple with at least 80 items is one pair, giving 17 pairs, 3,800 labelled items, and success rates from 8.0% to 79.0% (Table 11 in the appendix). The subject dictionaries carry the real model names, providers, release dates and harness names, so the budget-0 prior sees realistic inputs; the raw records are released with the code.

Synthetic IRT testbed. Because the real testbed is small (small models, three benchmarks), we complement it with a generator that follows a two-parameter item response model [Lord, 1980, Embretson and Reise, 2000]. Subject s has ability θ_s correlated with its visible attributes (release

Table 5: Brier ALC (lower is better) with 95% bootstrap CIs over pairs and pooled ECE on the two local testbeds, all methods evaluated on identical random-policy acquisition trajectories. Oracles are references, not predictors.

	Synthetic IRT (160 pairs)		Real Ollama (17 pairs)	
Method	ALC [95% CI]	ECE	ALC [95% CI]	ECE
constant 0.5	0.2500 [0.2500, 0.2500]	0.111	0.2500 [0.2500, 0.2500]	0.127
empirical mean (organisers)	0.2208 [0.2055, 0.2362]	0.114	0.2871 [0.2369, 0.3303]	0.170
attribute prior only	0.2148 [0.2040, 0.2251]	0.041	0.2340 [0.2198, 0.2468]	0.076
prior + shrinkage	0.1797 [0.1691, 0.1900]	0.035	0.2228 [0.1951, 0.2460]	0.063
shrinkage + ridge (always on)	0.1814 [0.1708, 0.1918]	0.029	0.2199 [0.1945, 0.2416]	0.061
shrinkage + kNN (always on)	0.1794 [0.1687, 0.1895]	0.033	0.2226 [0.1949, 0.2453]	0.061
full: LOO-gated ridge + kNN (rounds 1–3)	0.1797 [0.1691, 0.1900]	0.032	0.2224 [0.1949, 0.2454]	0.061
oracle: pair success rate	0.1598 [0.1483, 0.1710]	0.000	0.1961 [0.1675, 0.2208]	0.000
oracle: true item probability	0.1379 [0.1280, 0.1482]	0.007	—	—

year, reasoning effort, parameter count) plus noise; benchmark b has a location μ_b, a discrimination a_b and $K = 5$ topic clusters with difficulty offsets $c_k \sim \mathcal{N}(0, \sigma^2_{\text{topic}})$; item i in cluster $k(i)$ has difficulty $b_i = \mu_b + c_{k(i)} + \gamma\, z_{\text{len}}(i) + \varepsilon_i$ and $\Pr(y_{si} = 1) = \sigma(a_b(\theta_s - b_i))$. Item texts mix cluster-specific vocabulary (35% of tokens), benchmark template words, ambient vocabulary and numbers, so that lexical similarity carries difficulty information exactly when $\sigma_{\text{topic}} > 0$; half of the benchmarks expose a `tier=easy/medium/hard` feature derived from difficulty terciles. The main setting uses 10 subjects $\times$ 16 benchmarks $= 160$ pairs with 100–240 items each, $\sigma_{\text{topic}} = 0.70$ and $\gamma = 0.3$; a sweep varies both. Because the true probabilities are known, we can report two oracle references: the Brier score of the true item probabilities (the floor of any predictor) and that of the evaluation-set success rate used as a constant (the floor of any predictor without item-level information).

Experimental setup. Our rounds 1–3 harness reproduces the protocol of Section 2 with *per-pair* label lists (round 4 replaces this by the global list, Section 4): items of each pair are split 50/50 with a seeded shuffle, the acquisition pool is streamed in a seeded random order, the policy under test selects up to 31 labels, and the nested prefixes of that one trajectory are passed to every predictor for every evaluation item, so all methods are compared on identical labels. The evaluator's default policy is reproduced as accept-with-probability labels_remaining/items_remaining. We report Brier ALC with 95% bootstrap confidence intervals over pairs (1,000 resamples), paired bootstrap differences between methods (same pairs, same trajectories), per-budget Brier scores and pooled ECE. Baselines: the constant 0.5; the organisers' empirical mean (their code: 0.5 without matching labels, the raw mean otherwise); our prior alone; shrinkage alone; shrinkage with each item-level component always on; the LOO-gated model (shipped in rounds 1–3).

D Rounds 1–3: results of the per-pair predictor

Main comparison. Table 5 and Figure 2 give the headline numbers. On the real testbed the per-pair predictor (shipped in rounds 1–3) reaches 0.2224 [0.1949, 0.2454] against 0.2871 [0.2369, 0.3303] for the empirical mean and 0.25 for the constant; the paired difference to the empirical mean is -0.0647 [-0.0872, -0.0415] (significant; the full model is better on 82% of pairs) and to the constant -0.0276 [-0.0506, -0.0050] (significant). On the synthetic testbed the corresponding ALCs are 0.1797 [0.1691, 0.1900] versus 0.2208 and 0.25, with paired differences -0.0411 [-0.0488, -0.0339] and -0.0703 [-0.0805, -0.0592] (both significant). The empirical mean is *worse* than the constant at $n = 1$ on both testbeds ($B_1 = 0.330$ and 0.498) and only overtakes it from $n = 3$ (synthetic) and $n = 7$ (real); on the real testbed its ALC (0.2871) is *above* the constant's 0.25 (paired difference +0.0371 [-0.0081, +0.0779], not significant).

Where the gain comes from. Decomposing the ablation rows of Table 5: the prior alone improves on the constant by -0.0352 (synthetic) and -0.0160 (real) ALC; shrinkage adds -0.0351 [-0.0415, -0.0285] and -0.0113 [-0.0283, +0.0042] on top (the real-testbed difference is not significant with 17 pairs). The item-level components are a much smaller effect. Against shrinkage, always-on kNN changes the ALC by -0.0003 (synthetic) and -0.0002 (real), always-on ridge by +0.0017 and -0.0029,

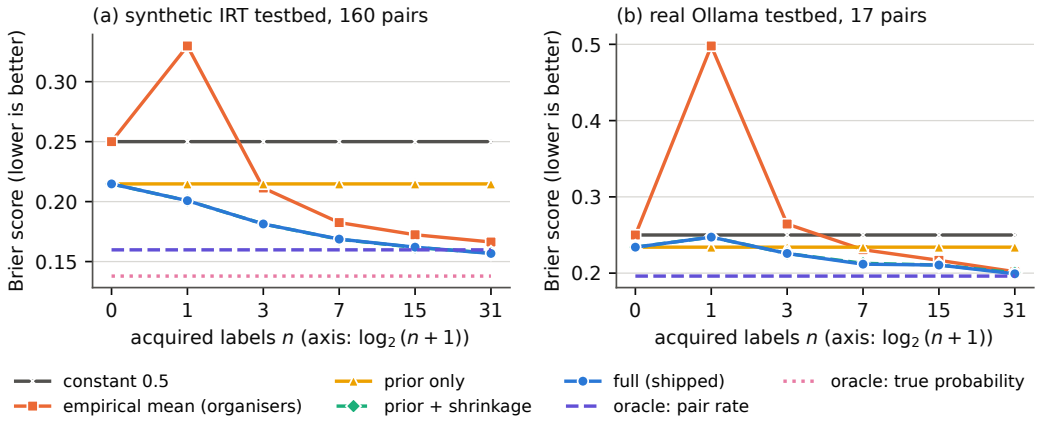


Figure 2: Brier score per budget (the ALC of Eq. 1 is the trapezoidal area under these curves on the $\log_2(n+1)$ axis). The empirical mean is worse than the constant at $n = 1$; shrinkage removes that penalty; the full model separates from shrinkage only at $n \geq 15$. Dashed/dotted lines are the oracle references.

and the LOO-gated full model by +0.0000 [-0.0002, +0.0003] (not significant) and -0.0004 [-0.0012, +0.0003] (not significant); at $n = 31$ alone the gated model changes the Brier score by -0.000 and -0.001. Replacing TF-IDF by `sentence-transformers/all-MiniLM-L6-v2` embeddings changes the full model's ALC by -0.0001 [-0.0005, +0.0001] (not significant), so we ship TF-IDF; Appendix E adds the LOO gate against always-on components and a sharper unigram TF-IDF, which makes the kNN worse. We report these as what they are: item-level adaptation from at most 31 labels is a marginal gain whose sign has to be protected by the gate.

Headroom. The oracle rows explain why. On the synthetic testbed, knowing the true probability of every item gives an ALC of 0.1379, knowing only the evaluation-set success rate 0.1598: perfect item-level discrimination is worth 0.0219 ALC, to be recovered from 15–31 labels at budgets that carry 30% of the weight. Shrinkage (0.1797) is 0.0199 above the rate oracle; that gap is the price of estimating a rate from few labels, concentrated at $n \leq 3$ (B_1: 0.041; B_{31}: -0.003), and only a better prior, i.e. training data, can reduce it. On the real testbed the gap is 0.0266 (0.2228 vs. 0.1961). The two parts of the headroom are of similar size but call for different remedies: a prior fitted to real subjects and benchmarks, and item difficulty information from outside the pair.

Prior strength and text signal. Figure 3 gives the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$ (shrinkage is best at 2 on synthetic and 8 on real; full-model range 0.0038 and 0.0182; we ship $\alpha_0 = 4$, the smallest worst-case regret and the best of $\{2, 4, 8\}$ on the training release) and the synthetic sweep over σ_{topic}, where the item-level effect stays two orders of magnitude below the item-level headroom (0.0080 to 0.0306): the gate protects against harm, it cannot create information.

Acquisition policies. None of the five policies we compared is significantly different from the evaluator's random policy on either testbed (Appendix E, Table 10): the novelty filter shipped in rounds 1–3 gives -0.0013 [-0.0038, +0.0013] (synthetic) and -0.0044 [-0.0108, +0.0025] (real), the organisers' uncertainty band +0.0001 and -0.0041. Selecting by $|p - 0.5|$ distorts the pair-rate estimate that carries most of the metric; we ship the policy closest to random that removes near-duplicates.

Frontier-like subjects. The official test set has subjects our testbeds lack (2025–2026 releases, long reasoning efforts, very large models, agent harnesses); over 160 such descriptions the budget-0 prior ranges from 0.39 to 0.82 (the cap; Appendix I). What a wrong prior costs is exact under shrinkage (Table 8): a prior of 0.7 on a pair whose true rate is 0.3 costs 0.062 ALC and loses to the constant (excess 0.040), whereas shrinkage beats the constant whenever $|r - 0.5| \geq 0.2$ and $|p_0 - r| \leq 0.3$. On the real testbed an optimistic shift of every subject's prior logit costs significantly ($+0.5$: +0.0134), a pessimistic one does not (-0.5: -0.0034, not significant); on the training release the hand-set prior averages 0.510 against a mean pair rate of 0.537, so the intercept stays at zero.

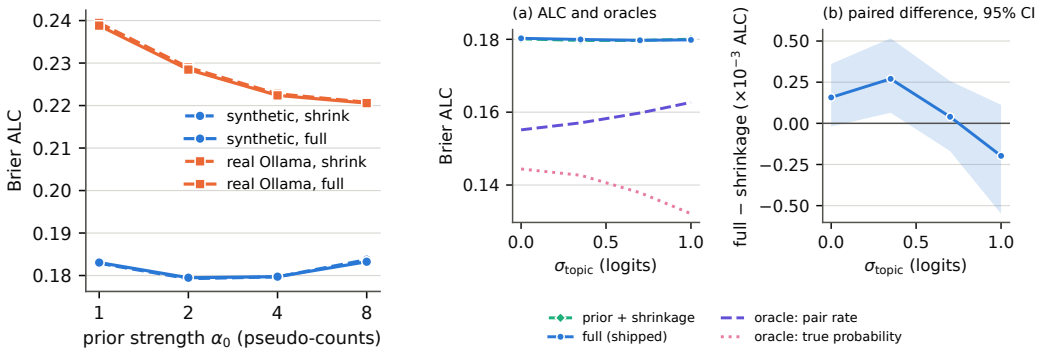


Figure 3: Left: Brier ALC vs. prior strength α_0 (dashed: shrinkage, solid: full model). Right: synthetic sweep of the topic-difficulty spread σ_{topic} at $\gamma = 0.3$: (a) ALC of shrinkage, the full model and the two oracles; (b) paired difference full − shrinkage with its 95% CI over pairs (negative favours the full model).

Replication and calibration. Appendix F reports replication over generator seeds and acquisition trajectories (the full model beats the empirical mean significantly in 4 of 4 further real trajectories). Pooled ECE (Table 5) is 0.032/0.061 (synthetic/real) for the full model against 0.114/0.170 for the empirical mean and 0.111/0.127 for the constant: shrinkage towards a sensible prior also fixes calibration at small budgets.

E Additional ablations

Prior strength. Table 14 gives the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$. The best value differs between testbeds (shrinkage: 2 on synthetic, 8 on real), reflecting the wider spread of pair rates in the synthetic generator; the ALC of the full model varies by 0.0038 (synthetic) and 0.0182 (real) over the range, so the choice matters at the third decimal. On the training release $\alpha_0 = 4$ gives 0.2051 against 0.2052 (2) and 0.2086 (8).

When does text help? The synthetic sweep (Figure 3, right; Table 13) varies how much difficulty is explained by topic. With no topic or length signal ($\sigma_{\text{topic}} = 0.00$, $\gamma = 0.0$) the full model's paired difference to shrinkage is +0.0002 [-0.0000, +0.0005], the gate's cost when there is nothing to find; at the strongest setting ($\sigma_{\text{topic}} = 1.00$) it is -0.0002 [-0.0005, +0.0001], while the item-level headroom (rate oracle minus true-probability oracle) grows from 0.0080 to 0.0306.

Gating. Replacing the LOO gate by always-on components at their default settings gives 0.1808 versus 0.1797 (synthetic; paired gated−always -0.0011 [-0.0017, -0.0005], significant) and 0.2203 versus 0.2224 (real; +0.0021 [-0.0009, +0.0048], not significant). The gate pays where always-on components overfit (the synthetic ridge) and costs a little where a component carries real signal (on the real testbed longer GSM8K problems are failed more often, which the always-on ridge exploits at $n \geq 15$); the second effect is within noise. Requiring 0, 1 or 2 standard errors of LOO improvement gives full-model ALCs of 0.1808, 0.1799 and 0.1797 (synthetic) and 0.2219, 0.2223 and 0.2224 (real); we ship the most conservative setting.

Sentence embeddings. The embeddings were computed offline for the real testbed (the model is on the validator's allow-list via `models.txt`, so it could have been shipped). They change the full model's ALC from 0.2224 to 0.2223 (paired -0.0001 [-0.0005, +0.0001], not significant) and the always-on kNN variant from 0.2226 to 0.2252 (+0.0026 [+0.0000, +0.0055], significant), at the cost of a neural dependency, model loading and about 10 ms per embedding on CPU.

Sharper lexical similarity does not help. A unigram-only TF-IDF whose target terms are restricted to the acquired vocabulary separates same-topic from other-topic synthetic items far better (mean cosine about 0.3 versus 0.1 in the generator) but makes the always-on kNN *worse*: paired against the shipped representation, the always-on kNN changes by +0.0009 [-0.0001, +0.0020] (synthetic)

and +0.0026 [+0.0006, +0.0050] (real), and the gated full model by +0.0004 [+0.0001, +0.0008] and +0.0006 [-0.0002, +0.0018]. With a handful of neighbours per topic the kernel-weighted residual is dominated by label noise; the bigram-diluted similarity acts as a regulariser. Item-level signal exists (the oracles show it) but 31 labels are too few to exploit it through similarity alone.

Acquisition policies. Table 10 compares policies for the full predictor on identical pools and streams. Plain stream order differs from the evaluator's random policy by -0.0013 [-0.0038, +0.0013] (synthetic, not significant) and -0.0044 [-0.0107, +0.0019] (real, not significant)—both are uniform samples, so this is the trajectory-to-trajectory noise floor of the comparison. The shipped novelty filter gives -0.0013 [-0.0038, +0.0013] and -0.0044 [-0.0108, +0.0025]. It does fire on the real testbed: on 10 of 17 pairs (40 candidates skipped in total, at most 10 per pair; templated BIG-Bench-Hard items such as *navigate* and *boolean expressions*, and SVAMP word problems sharing a template, reach cosine 0.98), and on none of the 160 synthetic pairs, whose pseudo-word items never get that similar—yet its ALC equals that of plain stream order to four decimals (0.2180 vs. 0.2180): skipping a near-duplicate changes which uniformly random items are labelled, not their distribution. The organisers' uncertainty band gives +0.0001 [-0.0027, +0.0030] and -0.0041 [-0.0096, +0.0016], and the novelty-plus-uncertainty variant -0.0013 and -0.0044: selecting by $|p - 0.5|$ reorders a near-random sample at small budgets and, once item-level corrections exist, biases it towards items of intermediate predicted difficulty, distorting the pair-rate estimate that carries most of the metric. None of the policies is significantly different from random on either testbed; we ship the one that is closest to random while removing redundant items.

F Replication

Regenerating the synthetic testbed with two further seeds gives full-model ALCs of 0.2266 and 0.1963 (empirical mean 0.2737 and 0.2438; shrinkage 0.2264 and 0.1963). On the real testbed, four further acquisition trajectories give full-model ALCs between 0.2127 and 0.2240 (empirical mean 0.2511–0.2841), and the full model beats the empirical mean significantly in 4 of 4 trajectories. A 3×3 grid over the kNN scale and anchor ($\kappa \in \{1, 2, 4\}$, $\tau \in \{1, 2, 4\}$) spans 0.1806–0.1809 on a smaller synthetic generator, i.e. the kernel constants matter little.

G Fitting pipeline details

`fit_prior_from_measurement_db.py` runs in five steps, each a sub-command. *plan*: list the repository, download the small `benchmarks` table of every directory and keep those with `response_type = binary` and `granularity = item` (the organisers' eligibility rule in `tools/prepare_data.py`); record the bytes still to download and stop above a cap (30 GB by default). *download*: fetch the eligible `subjects`, `items` and `responses` tables into the organisers' payload layout. *fit*: build one row per (subject, test condition, item) with the mean response over trials as label (fractional labels enter as two weighted rows), each pair receiving total weight one; compute the subject and item terms with the shipped `model.subject_terms` and `model.item_terms`, so that fitted coefficients map one-to-one onto the constants of Appendix B; add a one-hot for providers with at least 15 pairs and one for each training benchmark; fit an L_2-penalised logistic regression. Five-fold cross-validation holds out whole benchmarks; the penalty C and the logit cap are selected by the pair-weighted budget-0 Brier score of the prior on held-out pairs, and $\alpha_0 \in \{2, 4, 8\}$ by the closed-form shrinkage ALC of the protocol (50/50 split, 31 random acquisitions, Eq. 1) on held-out pairs with at least 80 items, paired against the hand-set prior on the same pairs. *testbeds*: evaluate the fitted table inside the harness on both local testbeds against the shipped constants. *dry-run*: generate tables with the release schema (shared pool of subjects, random subsets per benchmark, benchmark difficulty offsets with standard deviation one logit, labels from a known coefficient table over the prior's own terms, two ineligible benchmarks) and run *fit* on them. The dry run used 14 eligible benchmarks, 299 pairs and 47,784 rows; a light-regularisation fit recovers all 13 slope signs with maximum error 0.05 and mean error 0.02 logits, and the fitted benchmark effects have standard deviation 0.57. Its cross-validated selection chose $C = 0.003$ and $\alpha_0 = 4$ and did not beat the hand-set prior (budget-0 Brier +0.0036 [+0.0004, +0.0069]; shrinkage ALC +0.0013 [+0.0001, +0.0027]), so on these tables the rule would have kept the hand-set constants.

The run on the release. Revision `54074453da`; 163 eligible directories, 144 with at least one pair of 20 or more distinct items; the pair unit is (benchmark directory, subject, test condition), since the release records several test conditions per subject on many benchmarks; 16,718 pairs, 3,118,742 rows (each pair capped at 200 items); 22 providers with at least 15 pairs received a coefficient. Pair success rates have mean 0.537 and standard deviation 0.265 across pairs (benchmark means: standard deviation 0.223), while the hand-set p_0 has mean 0.510 and standard deviation 0.072. The penalty grid ran from $C = 0.001$ to 1; the selection took $C = 0.001$, cap 1.5 and $\alpha_0 = 2$ (cross-validated budget-0 Brier 0.2421, shrinkage ALC 0.2061). The fitted benchmark effects have standard deviation 0.46 logits. Fitted vs. effective hand-set coefficients: release year +0.14 vs. +0.24 per year, size +0.08 vs. +0.17 per doubling, hard-tier cue -0.65 vs. -0.60, easy-tier cue +0.08 vs. +0.50, item length +0.08 vs. +0.12; the reasoning-effort and harness terms shrink to about zero (-0.08, -0.02; 58% of pairs carry no provider field and most carry no effort or harness field), and the provider table spans -0.23 (`hugging face`) to +0.55 (`unknown`). Inside the harness on the local testbeds the fitted table changes the full model's ALC by +0.0021 [-0.0006, +0.0045] (real) and +0.0058 [+0.0032, +0.0081] (synthetic); the result files are `prior_fit.json` and `prior_fit_testbeds.json`, and the ship rule's verdict is recorded there (not shipped).

H Item-text difficulty prior: details

Pre-registration and outcome. `experiments/results/round3/PREREG.md` (SHA-256 prefix `bb54af425d`, recorded in every result file and verified unchanged at build time) fixed features, target, model, nested selection, evaluation and ship rule before any text model was fitted. `PREREG_ADDENDUM_1.md` (verified unchanged) registered the two exploratory variants after the primary folds had run, explicitly without influence on the decision. The rule: ship iff (a) the benchmark-held-out ALC difference (text minus hand-set) has a 95% cluster-bootstrap CI below zero, (b) the real testbed is not significantly worse, and the engineering gates hold (≥ 200 calls/s on one core with a new target item per call, weights ≤ 5 MB, validator and smoke test pass, never raises). Outcome: (a) fails, (b) holds, gates pass; the text prior is not shipped.

Features and target. JSON contents (the release stores many items as JSON objects) are flattened into `key: value` lines; words and word bigrams come from the first 2,000 characters (digit runs mapped to a length token), character trigrams from the first 500, and `item_features` tokens form a fourth family; each family is sublinear-tf weighted and ℓ_2-normalised, then hashed with `zlib.crc32` (sign from bit 31) into 262,144 buckets. The target of item i is $\operatorname{logit} \frac{n_i r_i + 0.5}{n_i + 1}$ minus the mean hand-set subject logit of the n_i pairs that evaluated it and minus its hand-set item logit, clipped to ± 5. At most 1,000 items per benchmark (109,233 in total) are used for fitting; all 515,067 items of the frame are scored (237,628,268 non-zeros). Exact duplicates of a test item's content in a training benchmark are rare (0.02% of frame items on average), so shared items do not inflate the cross-validation.

Nested selection and fits. In each outer fold a four-fold benchmark-grouped inner CV chooses the ridge penalty from $\{0.03, 0.1, 0.3, 1, 3, 10\}$ by weighted MSE of the target; the inner out-of-fold scores give W, B (pair-weighted logistic likelihood), the cap ($\{1.5, 2.5, 3.5\}$, training budget-0 Brier) and the within-pair weight λ of the text offset ($\{0, 0.5, 1\}$, training closed-form ALC). Selected: penalty 10 in every fold (the grid maximum), caps 1.5, 2.5, $\lambda = 0$, W from -0.32 to -0.03 and B from +0.06 to +0.21. The held-out R^2 of the residual target was -0.06 to 0.04 across folds (inner CV: -0.04 to -0.00). The final fit on all 144 benchmarks again selected penalty 10 with $W = -0.20$, $B = +0.14$, cap 1.5 and $\lambda = 0$; it is kept as `text_prior_candidate.json` (1.40 MB) and not shipped.

Results. Table 6. Per budget the paired differences are -0.0000 [-0.0025, +0.0027] at $n = 1$ and -0.0000 [-0.0013, +0.0014] at $n = 3$ (gap to the pair-rate oracle closed: 0.1% [-5.6, 6.0] and 0.1% [-4.2, 4.3]); budget-0 Brier changes by +0.0003 [-0.0036, +0.0046]; the plain pair bootstrap gives -0.0000 [-0.0001, +0.0001]. The text prior has the lower mean ALC in 1 of 5 folds (significantly in 0) and on 57 of 135 benchmarks. Refitting only the intercept shift ($W = 0$; a descriptive decomposition) gives +0.0003 [-0.0007, +0.0009]: the hand-set prior's level is already right on held-out benchmarks. With the text prior, $\alpha_0 = 2$ gives +0.0001 and $\alpha_0 = 8$ +0.0035 (significant) relative to the hand-set prior at $\alpha_0 = 4$. The exploratory variants (E1: grid 10, 100, 1000, 10000, selected 100, 1000, 10000; E2: statistics only, grid 0.1, 1, 10, 100, selected 100) changed the ALC by +0.0002 [-0.0007,

Table 6: Item-text prior: benchmark-held-out cross-validation on the training release (closed-form protocol, shrinkage with $\alpha_0 = 4$ unless stated, no item-level components). B_0 over all items of each of the 16,718 pairs; B_1–B_7 and ALC on the evaluation half of the 16,303 pairs with at least 80 items. Exploratory rows were registered after the primary result; the oracles are references.

Method	B_0	B_1	B_3	B_7	ALC
hand-set prior (shipped)	0.2388	0.2235	0.2083	0.1956	0.2051
hand-set + item-text prior (pre-registered)	0.2391	0.2234	0.2083	0.1956	0.2051
same, $\alpha_0 = 2$	0.2391	0.2240	0.2086	0.1955	0.2052
same, $\alpha_0 = 8$	0.2391	0.2278	0.2141	0.2002	0.2086
exploratory E1: penalty up to 10^4	0.2397	0.2239	0.2085	0.1957	0.2053
exploratory E2: 16 statistics only	0.2395	0.2237	0.2084	0.1956	0.2052
hand-set + fitted intercept shift only	0.2398	0.2240	0.2086	0.1957	0.2054
intercept only (training mean)	0.2519	0.2320	0.2127	0.1975	0.2096
oracle: held-out benchmark mean	0.2187	0.2115	0.2022	0.1932	0.1990
oracle: pair success rate	–	0.1756	0.1756	0.1756	–

+0.0011] and +0.0001 [-0.0010, +0.0014] and again had negative W in 5 and 5 of 5 folds. The held-out benchmark-mean oracle closes 30%, 25% and 19% of the gap at $n = 0, 1, 3$.

Local testbeds. Full harness, identical trajectories: real ALC 0.2224 (hand-set) vs. 0.2243 (text), paired +0.0019 [-0.0002, +0.0037], text better on 35% of pairs; without the BIG-Bench-Hard pairs, whose items may overlap the `fewshot_ttt_bbh` training benchmark, +0.0017 [-0.0007, +0.0042] (13 pairs). Synthetic: 0.1797 vs. 0.1800 (+0.0003 [-0.0002, +0.0008]); its pseudo-word items carry no transferable text signal by construction.

Engineering gates and the shipped file. With the candidate loaded, `predict` runs at 3,056, 673 and 417 calls/s on one core with a new 15-, 300- and 2,000-word target item per call at budget 31 (the per-item text score is cached), the weights load in 22 ms, and the smoke test passes 44 cases, including malformed JSON contents and a corrupted weight specification (which falls back to the hand-set prior); standard-library and numpy scores agree to 9.0×10^{-9}. Since the rule failed, the round-3 submission ZIP was unchanged (SHA-256 prefix `3f0d15c879d9`); the development `model.py` kept the text-prior hooks, which are inactive without a weight file and reproduced the then-shipped file's predictions bit for bit on 5,544 of 5,544 test cases.

Benchmarks of the training release used (144). `abstract_reason`, `adaptivestep`, `advprompter`, `aegis`, `afrieval`, `afrimedqa`, `agentic_review_perturbation`, `agi_elo`, `ai2d_test`, `aider`, `algotune`, `alignment_faking`, `alpha_sql`, `appworld`, `auditing_sabotage_bench`, `averimatec`, `babilong`, `bbq`, `beavertails`, `benger`, `bertaqa`, `bird_sql`, `brace`, `braveguard`, `bridging_gap`, `causal_axioms`, `cellverse`, `ceval`, `chi_bench`, `clasheval`, `cmmlu`, `critic_discernment_game`, `crow`, `cruxeval`, `dave`, `devbench`, `editbench`, `edu_circuit_hw`, `egoschema`, `ehrflowbench`, `elicitation_game`, `emoji_attack`, `engdesign`, `enginemt_qa`, `erbench`, `evouna`, `exploitgym`, `faithcot`, `felm`, `fewshot_ttt_bbh`, `financebench`, `finegrain_t2i`, `haiid`, `hallusionbench`, `harmbench`, `healthadminbench`, `helm_afr`, `helm_bbq`, `helm_thaiexam`, `hle`, `icpc2_code_selector`, `igakuqa119`, `ikp`, `interchangeable_token_embeddings`, `jailbreakbench`, `judgetuning`, `kernelbench`, `kmmlu`, `kormedmcqa`, `legal_rag_bench`, `legalbench`, `lingoly`, `liveaopsbench`, `llm_bp_tag`, `llm_uncertainty_bench`, `llmail_inject`, `llms_lean_formalization`, `mathconstraint`, `mathvista_mini`, `medhelm_ehr_sql`, `medhelm_head_qa`, `medhelm_med_mcqa`, `medhelm_med_qa`, `medhelm_medbullets`, `medhelm_medcalc_bench`, `medhelm_medec`, `medhelm_medhallu`, `medhelm_pubmed_qa`, `medhelm_race_based_med`, `mind2web`, `mj_bench`, `mmbench_v11`, `mmlu`, `mmlupro`, `monkey_power_laws`, `morphkv`, `multi_swebench`, `mypcbench`, `naep_llm_students`, `naturebench`, `nester`, `nyu_ctf_bench`, `ood_prediction`, `openbiorq`, `os_harm`, `perfcodebench`, `perumedqa`, `planbench`, `proofaug`, `prox`, `pxplore`, `real_webagents`, `realtimeqa`, `reliancescope`, `researchcodebench`, `revolve`, `risebench`, `robust_reasoning_benchmark`, `safeagentbench`, `sgrades`, `sib200`, `sketchjudge`, `software_world_models`, `statqa`, `subjective_qa`, `sugarcrepe`, `sustainable_food`, `swe_rebench`, `swe_smith`, `swe_together`, `swebench_live`, `swepolybench`, `tau2_bench`, `tensortrust`, `truthfulqa_mc`, `visual_memory`, `visual_riddles`, `visualwebarena`, `vl_rewardbench`, `weavebench`, `wikihow_agent`, `workarena`, `workbench_revisited`, `worldcentralbanks`.

I Prior sensitivity for frontier-like subjects

Table 7 gives the budget-0 prior p_0 of the shipped constants for subject descriptions that the official test set is likely to contain and our testbeds do not. Table 8 gives the exact expected cost of a miscalibrated prior under the per-pair shrinkage rule, and Table 9 the measured effect of shifting every subject's prior logit on the two testbeds (`PRIOR_INTERCEPT` in `model.py`; all other constants unchanged; identical trajectories; generated by `prior_sensitivity.py`).

Discussion. The official test set has subjects our testbeds lack (2025–2026 releases, long reasoning efforts, very large models, agent harnesses). Appendix I tabulates the budget-0 prior over 160 such descriptions: p_0 ranges from 0.39 to 0.82 (the cap); the organisers' GPT-4o and 2026 medium-effort sample subjects receive 0.49 and 0.66, our testbed models 0.36–0.39. What a wrong prior costs is exact: with $\alpha_0 = 4$ and n labels the expected Brier excess over the pair-rate oracle is $\mathbb{E}[(\rho_n - r)^2]$ with binomial successes (Table 8). A prior of 0.7 on a pair whose true rate is 0.3 costs 0.062 ALC and loses to the constant (excess 0.040); shrinkage beats the constant whenever $|r - 0.5| \geq 0.2$ and $|p_0 - r| \leq 0.3$, and never when $|r - 0.5| < 0.15$, where the constant is close to optimal. Shifting every subject's prior logit by ± 0.5 or ± 1 (Table 9) is asymmetric on the real testbed: $+0.5$ and $+1$ cost +0.0134 and +0.0353 ALC (both significant), whereas -0.5 changes it by -0.0034 [-0.0117, +0.0049] (not significant)—small models at 8.0–79.0% accuracy tolerate a pessimistic prior; on the synthetic testbed zero is best and both ± 0.5 shifts cost significantly (+0.0037, +0.0047). On the training release the hand-set prior averages 0.510 against a mean pair rate of 0.537, i.e. it is not optimistic for real systems. We keep the intercept at zero and name it as the first constant to change if the official formative score points the other way.

Table 7: Budget-0 prior p_0 (no item cues; 600-character item) by release year and `reasoning_effort` for a subject with no size or name cue on the default harness (left), and for a high-effort subject on a non-default harness with a *mini* name cue, a 70B or a 400B parameter count (right). The organisers' sample subjects receive 0.49 (GPT-4o, 2024-05) and 0.66 (2026-01, medium effort, non-default harness).

	default harness, no name cue				high effort, non-default harness		
Release year	none	low	medium	high	mini	70B	400B
2023	0.44	0.46	0.49	0.51	0.48	0.58	0.69
2024	0.50	0.52	0.55	0.58	0.54	0.64	0.74
2025	0.56	0.58	0.61	0.63	0.60	0.70	0.78
2026	0.62	0.64	0.67	0.69	0.66	0.75	0.82

Table 8: Exact expected Brier ALC excess over the pair-rate oracle $r(1-r)$ of the per-pair shrinkage rule ($\alpha_0 = 4$, clipping to $[0.01, 0.99]$) when the prior rate is p_0 and the true pair rate is r; the number of successes among the n acquired labels is binomial(n, r) and the ALC weights are those of Eq. 1. The last column is the excess of the constant 0.5; † marks cells where the constant is better.

	prior rate p_0						
True rate r	0.3	0.4	0.5	0.6	0.7	0.8	constant 0.5
0.2	0.010	0.020	0.036	0.060	0.089	0.126†	0.090
0.3	0.009	0.012	0.022	0.039	0.062†	0.091†	0.040
0.4	0.013†	0.010†	0.013†	0.023†	0.040†	0.063†	0.010
0.5	0.024†	0.014†	0.011†	0.014†	0.024†	0.040†	0.000
0.6	0.040†	0.023†	0.013†	0.010†	0.013†	0.023†	0.010
0.7	0.062†	0.039	0.022	0.012	0.009	0.012	0.040
0.8	0.089	0.060	0.036	0.020	0.010	0.007	0.090

J Validator and smoke-test output

The organisers' validator exits with code 0 and prints

```
OK: paiec_submission.zip looks like a valid submission ZIP.
```

Table 9: Brier ALC of the prior alone, shrinkage and the full model when every subject's prior logit is shifted by `PRIOR_INTERCEPT` (random policy, same trajectories as Table 5). The shipped value is 0.

Shift (logits)	Real Ollama (17 pairs) prior	shrinkage	full	Synthetic IRT (160 pairs) prior	shrinkage	full
-1.0	0.2511	0.2229	0.2229	0.2689	0.1959	0.1955
-0.5	0.2319	0.2191	0.2190	0.2286	0.1835	0.1834
$+0.0$	0.2340	0.2228	0.2224	0.2148	0.1797	0.1797
$+0.5$	0.2663	0.2364	0.2358	0.2268	0.1844	0.1844
$+1.0$	0.3250	0.2585	0.2576	0.2527	0.1932	0.1932

(full output in `experiments/results/validator_output.txt`; ZIP SHA-256 prefix `c76331d084f1`). The smoke test (`experiments/results/smoke_test.json`) records 44 successful cases and the timings of the per-pair path quoted in Appendix B.

K Additional tables

Table 10: Acquisition policies (full predictor): Brier ALC [95% CI] and paired difference to the evaluator's random policy on the same pairs (negative favours the policy). The BLE rule accepts a candidate when its current prediction satisfies $|p - 0.5| \leq 0.15$ [Truong et al., 2026].

	Synthetic		Real	
Policy	ALC [95% CI]	Δ vs random [95% CI]	ALC [95% CI]	Δ vs random [95% CI]
random (evaluator default)	0.1797 [0.1691, 0.1900]	—	0.2224 [0.1949, 0.2454]	—
stream order	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1915, 0.2400]	-0.0044 [-0.0107, 0.0019]
novelty filter (rounds 1–3)	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1912, 0.2407]	-0.0044 [-0.0108, 0.0025]
novelty + uncertainty	0.1785 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1912, 0.2407]	-0.0044 [-0.0108, 0.0025]
uncertainty band (BLE rule)	0.1798 [0.1693, 0.1896]	+0.0001 [-0.0027, 0.0030]	0.2183 [0.1905, 0.2409]	-0.0041 [-0.0096, 0.0016]

Table 11: Real Ollama testbed: one pair per (model, harness, benchmark). Accuracy is the success rate over all items of the pair (acquisition and evaluation pools together).

Model (Ollama tag)	Harness	Benchmark	Items	Acc. (%)
qwen2.5:1.5b-instruct	8-shot CoT	GSM8K (test, first 200)	200	37.5
qwen2.5:3b-instruct	8-shot CoT	GSM8K (test, first 200)	200	24.0
qwen2.5:7b-instruct	8-shot CoT	GSM8K (test, first 200)	200	42.5
qwen2.5:1.5b-instruct	8-shot CoT	SVAMP (test, first 200)	200	79.0
qwen2.5:3b-instruct	8-shot CoT	SVAMP (test, first 200)	200	67.0
llama3.2:3b	direct answer	BBH mix (10 subtasks $\times$ 30)	300	49.3
qwen2.5:1.5b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	39.3
qwen2.5:3b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	29.3
qwen2.5:7b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	51.0
llama3.2:3b	direct answer	GSM8K (test, first 200)	200	8.0
qwen2.5:1.5b-instruct	direct answer	GSM8K (test, first 200)	200	12.0
qwen2.5:3b-instruct	direct answer	GSM8K (test, first 200)	200	12.5
qwen2.5:7b-instruct	direct answer	GSM8K (test, first 200)	200	23.5
llama3.2:3b	direct answer	SVAMP (test, first 200)	200	33.0
qwen2.5:1.5b-instruct	direct answer	SVAMP (test, first 200)	200	32.5
qwen2.5:3b-instruct	direct answer	SVAMP (test, first 200)	200	39.5
qwen2.5:7b-instruct	direct answer	SVAMP (test, first 200)	200	64.5

Table 12: Brier score per budget for the main methods (random policy). The last column is the ALC of Eq. 1.

Testbed	Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
synthetic	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.330	0.211	0.183	0.172	0.166	0.2208
	attribute prior only	0.215	0.215	0.215	0.215	0.215	0.215	0.2148
	prior + shrinkage	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	full (shipped)	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	oracle: pair success rate	0.160	0.160	0.160	0.160	0.160	0.160	0.1598
real	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.498	0.264	0.231	0.217	0.202	0.2871
	attribute prior only	0.234	0.234	0.234	0.234	0.234	0.234	0.2340
	prior + shrinkage	0.234	0.247	0.226	0.213	0.211	0.201	0.2228
	full (shipped)	0.234	0.247	0.226	0.212	0.211	0.199	0.2224
	oracle: pair success rate	0.196	0.196	0.196	0.196	0.196	0.196	0.1961

Table 13: Synthetic signal sweep (160 pairs per setting, seed 0): ALC of shrinkage and of the full model, their paired difference, and the two oracle references.

σ_{topic}	γ	Shrinkage	Full [95% CI]	Full − shrink [95% CI]	Oracle rate	Oracle p
0.00	0.0	0.1816	0.1818 [0.1708, 0.1926]	+0.0002 [-0.0000, 0.0005]	0.1543	0.1464
0.00	0.3	0.1801	0.1803 [0.1692, 0.1906]	+0.0002 [-0.0000, 0.0004]	0.1551	0.1444
0.35	0.3	0.1797	0.1800 [0.1689, 0.1905]	+0.0003 [0.0001, 0.0005]	0.1571	0.1427
0.70	0.3	0.1797	0.1797 [0.1691, 0.1900]	+0.0000 [-0.0002, 0.0003]	0.1598	0.1379
1.00	0.3	0.1800	0.1798 [0.1694, 0.1898]	-0.0002 [-0.0005, 0.0001]	0.1627	0.1321
0.70	0.0	0.1793	0.1796 [0.1688, 0.1896]	+0.0002 [0.0000, 0.0005]	0.1583	0.1392

Table 14: Sensitivity to the prior strength α_0 (Brier ALC).

	$\alpha_0 = 1$	$\alpha_0 = 2$	$\alpha_0 = 4$	$\alpha_0 = 8$
Synthetic, shrinkage	0.1829	0.1793	0.1797	0.1837
Synthetic, full	0.1831	0.1795	0.1797	0.1832
Real, shrinkage	0.2394	0.2290	0.2228	0.2208
Real, full	0.2388	0.2285	0.2224	0.2206